Université de Toulouse

Master1 SME

Compte Rendu

Mesure de la température et de l'humidité par capteur DHT11 sur STM32

Réalisé par : Sarah KISSOUM, Fatima Zahra BENMERIEME

Encadrant : *Thierry PERISSE*

Année universitaire **Date : Février 2026** **2025/2026**

Sommaire :

Introduction :.....3

I. Matériel et Principe de Fonctionnement 3

1.1. Matériel utilisé 3

1.2. Principe du capteur DHT11.....4

1.3. Brochage du capteur 5

II. Implémentation technique sur STM32.....5

2.1. Schéma du câblage 5

2.2. Gestion du Timer pour la mesure en Microsecondes 6

2.3. Description de la communication avec le capteur 6

2.4. Affichage sur l'Ecran LCD I2C 9

III. Résultats Expérimentaux et Analyse.....9

3.1. Validation de l'impulsion de démarrage.....10

3.2. Analyse de la trame du capteur (40 bits).....11

Conclusion.....12

Introduction :

Ce rapport présente la conception et la réalisation d'un système d'acquisition de données, portant sur la mesure de la température et de l'humidité. Le projet utilise une carte NUCLEO-L476RG de STMicroelectronics, d'un capteur numérique DHT11 connecté via le système Grove, et d'un écran Grove LCD RGB pour afficher les mesures en temps réel. L'objectif principal de ce TP est de comprendre et mettre en œuvre la chaîne de mesure numérique complète. Cela passe par l'envoi du signal de démarrage depuis le microcontrôleur, puis par la lecture et le décodage des données renvoyées par le DHT11, avant de les afficher sur le LCD. Les signaux électriques échangés entre le STM32 et le capteur DHT11 ont été analysés à l'aide d'un PicoScope pour vérifier que les signaux correspondent bien au protocole du DHT11 et que le système fonctionne correctement.

I.Matériel et Principe de Fonctionnement

1.1 Matériel Utilisé

• Carte de développement : **STMicroelectronics NUCLEO-L476RG**

• Capteur de température et d'humidité V1.0 : **DHT11**

• Afficheur: **Seeed Studio Grove LCD RGB (I2C)**

• Oscilloscope : **PicoScope** pour l'analyse des signaux

1.2 Principe du Capteur DHT11

Le capteur DHT11 fonctionne en détectant les changements de température et d'humidité à l'aide d'éléments de détection internes. Ces signaux sont traités par une puce interne et convertis en données numériques.Le capteur envoie ensuite les relevés de température et d'humidité à un microcontrôleur via une interface numérique monofilaire

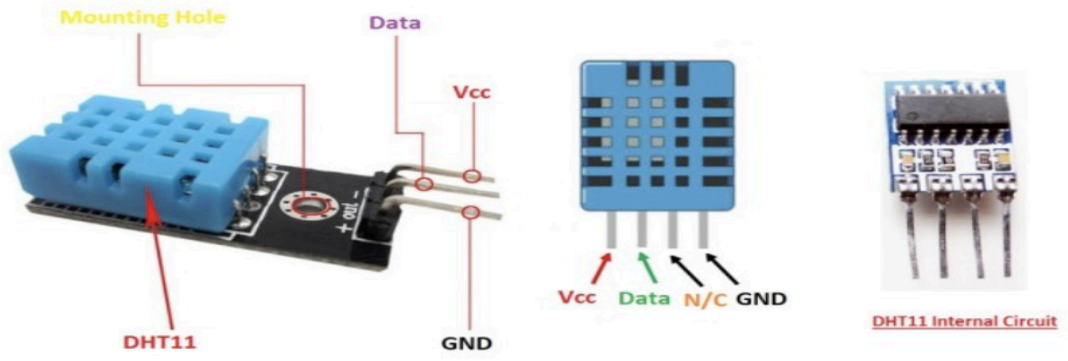

Figure 1 : Principe de fonctionnement du DHT11

Le fonctionnement du capteur complet se fait en 3 étapes :

1. **Déclenchement (Start Signal) :** Le microcontrôleur envoie un signal de démarrage en tirant la ligne de données à l'état bas pendant au moins 18 ms, puis la relâche.
2. **Réponse du capteur :** Le DHT11 détecte le signal de démarrage et répond en tirant la ligne à l'état bas pendant 80 µs, puis à l'état haut pendant 80 µs, confirmant sa disponibilité pour la transmission.
3. **Transmission des données :** Le capteur envoie la trame de 40 bits contenant les mesures et la somme de contrôle (checksum).

Le format des données sur 40 bits est structuré comme suit :

Position	Taille	Contenu
Bits 1–8	8 bits	partie entière de l'humidité relative (% HR)
Bits 9–16	8 bits	Partie décimale de l'humidité relative
Bits 17–24	8 bits	Partie entière de la température (°C)
Bits 25–32	8 bits	Partie décimale de la température
Bits 33–40	8 bits	Somme de contrôle (Checksum)

La vérification de l'intégrité des données se base sur le calcul du checksum : si la somme des quatre premiers octets est égale au cinquième octet, les données sont considérées comme valides.

Dans le cas contraire, la lecture est rejetée, pour ne pas afficher des valeurs fausses causées par des interférences ou des erreurs de transmission.

1.3 Brochage du capteur

La broche de signal (SIG) sert à la fois à recevoir la requête de démarrage du STM32 et à envoyer la trame de 40 bits en retour. Une résistance de tirage vers le haut (pull-up) est généralement intégrée sur le module Grove pour maintenir la ligne à l'état haut lorsqu'elle est inactive, garantissant un état défini en l'absence de communication.

Broche	Nom	Description
1	GND	Masse (0V)
2	VCC	Alimentation (3.3V à 5V)
3	NC	Non connecté
4	SIG	Signal de données bidirectionnel (single-wire)

II. Implémentation technique sur STM32

La mise en œuvre de ce projet sur la carte NUCLEO-L476RG nécessite une gestion précise des temporisations et des changements d'état des broches GPIO du microcontrôleur STM32L476RG. L'environnement de développement utilisé est STM32CubeIDE, qui inclut la bibliothèque HAL (Hardware Abstraction Layer).

II.1 Schéma du câblage

Le montage physique a été réalisé en connectant les différents modules Grove à la carte NUCLEO via un connecteur Grove. Les connexions sont les suivantes :

- **Capteur DHT11 :**
  - **GND** sur une broche **GND de la Nucleo**.
  - **VCC** sur la broche **5V**.
  - **SIG** sur la broche **PA9**.
- **Écran LCD :**
  - **GND** sur une broche **GND**.

- **VCC** sur la broche **5V**.
- **SDA** sur la broche **PB9** (**I2C1_SDA**).
- **SCL** sur la broche **PB8 (I2C1_SCL)**.

Le câblage est représenté en figure 4 ci-dessous.

II.2 Gestion du Timer pour la mesure en microsecondes

Le protocole du DHT11 dépend entièrement du timing. La distinction entre un bit '0' et un bit '1' se fait par la mesure de la durée à l'état haut du signal. Il est donc impératif de disposer d'une base de temps précise à la microseconde près, ce que HAL_Delay() ne permet pas, car elle fonctionne en millisecondes.

Un timer matériel du STM32 (le TIM6 comme visible dans le code source) a été configuré pour s'incrémenter chaque microseconde. La fonction Delay_us() a été développée à partir de ce timer :

```
// Déclaration externe du timer utilisé pour les délais microsecondes
extern TIM_HandleTypeDef htim6;

// Fonction de délai en microsecondes en utilisant le timer TIM6
void Delay_us(uint16_t us)
{
    __HAL_TIM_SET_COUNTER(&htim6, 0); // Réinitialiser le compteur du timer
    while (__HAL_TIM_GET_COUNTER(&htim6) < us); // Attendre jusqu'à atteindre le nombre de microsecondes
}
```

Cette fonction réinitialise le compteur du timer à zéro, puis attend en boucle active jusqu'à ce que le compteur atteigne la valeur (us) souhaitée, ce qui donne une précision suffisante pour bien suivre le protocole du DHT11.

II.3 Description Détaillée de la Communication avec le Capteur

La communication avec le DHT11 s'effectue sur une seule broche (PA9) qui doit changer dynamiquement de direction (sortie puis entrée) au cours de la transaction. Le code source du projet, visible dans les captures d'écran de STM32CubeIDE, illustre cette implémentation.

- Phase 1 Signal de démarrage (DHT11_Start) :

La broche PA9 est d'abord configurée en sortie (mode Open-Drain). Le STM32 tire la ligne à l'état bas pendant 18 ms, puis la relâche. Après un délai de (20 -40) µs, la broche est reconfigurée en entrée pour permettre la lecture de la réponse du capteur.

```c
// Fonction pour initier la communication avec le capteur DHT11
void DHT11_Start(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    // Configurer PA9 en sortie open-drain pour envoyer le signal au capteur
    GPIO_InitStruct.Pin = GPIO_PIN_9;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_OD;
    GPIO_InitStruct.Pull = GPIO_PULLUP; // Pull-up nécessaire pour DHT11
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    // Tirer la ligne à 0 pendant 18 ms pour signaler le début de communication
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
    HAL_Delay(18);

    // Relâcher la ligne (mettre à 1) pendant 20-40 µs
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);
    Delay_us(30);

    // Configurer PA9 en entrée pour recevoir la réponse du capteur
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}
```

Figure 1 : Code source de la fonction DHT11_Start dans STM32CubeIDE

- Phase 2 Vérification de la réponse (DHT11_CheckResponse) :

Après le signal de démarrage, la fonction DHT11_CheckResponse() attend la réponse du capteur. Elle attend d'abord 40 µs, puis vérifie que la ligne est à l'état bas (début de la réponse du capteur). Si c'est le cas, elle attend 80 µs supplémentaires et vérifie que la ligne est revenue à l'état haut, confirmant la présence et la disponibilité du capteur.

```c
// Fonction pour vérifier la réponse du capteur DHT11
uint8_t DHT11_CheckResponse(void)
{
    uint8_t response = 0;

    Delay_us(40); // Attendre que le capteur tire la ligne à 0
    if (!HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9)) // Vérifier si le capteur a tiré la ligne à 0
    {
        Delay_us(80); // Attendre la fin du signal de réponse
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9)) response = 1; // Si la ligne est repassée à 1, réponse valide
    }

    // Attendre la fin de la transmission du signal du capteur
    while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9));
    return response;
}

// Lire un bit envoyé par le DHT11
uint8_t DHT11_ReadBit(void)
{
    while (!HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9)); // Attendre que la ligne passe à 1
    Delay_us(40); // Attendre 40 µs pour mesurer la durée du signal

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9)) // Si la ligne est encore à 1 après 40 µs, le bit est 1
    {
        while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9)); // Attendre la fin du signal
        return 1;
    }
    else
    {
        return 0; // Sinon le bit est 0
    }
}
```

Figure 2 : Code source des fonctions DHT11_CheckResponse et DHT11_ReadBit

- Phase 3 Lecture des données (DHT11_ReadData) :

La fonction DHT11_ReadBit() est appelée 40 fois pour lire chaque bit de la trame. Pour chaque bit, elle attend la fin de l'état bas de 50 µs, puis mesure la durée de l'état haut. Si la ligne est encore haute après 40 µs, le bit est considéré comme un '1' logique ; sinon, c'est un '0' logique. Les 40 bits sont ensuite assemblés en 5 octets par la fonction DHT11_ReadByte(), puis interprétés dans DHT11_ReadData().

```c
// Lire un octet (8 bits) envoyé par le DHT11
uint8_t DHT11_ReadByte(void)
{
    uint8_t i, byte = 0;
    for (i = 0; i < 8; i++)
    {
        byte <<= 1; // Décaler le byte de 1 pour préparer le bit suivant
        byte |= DHT11_ReadBit(); // Ajouter le bit lu
    }
    return byte;
}

// Lire les données de température et d'humidité du DHT11
void DHT11_ReadData(uint8_t *humidity, uint8_t *temperature)
{
    uint8_t hum_int, hum_dec, temp_int, temp_dec, checksum;

    DHT11_Start(); // Initier la communication
    if (DHT11_CheckResponse()) // Vérifier si le capteur répond
    {&&&@&
        // Lire les 5 octets envoyés par le capteur
        hum_int = DHT11_ReadByte();  // Humidité entière
        hum_dec = DHT11_ReadByte();  // Humidité décimale (toujours 0 pour DHT11)
        temp_int = DHT11_ReadByte(); // Température entière
        temp_dec = DHT11_ReadByte(); // Température décimale (toujours 0 pour DHT11)
        checksum = DHT11_ReadByte(); // Octet de contrôle pour vérifier l'intégrité

        // Vérifier que la somme des 4 premiers octets correspond au checksum
        if (checksum == (hum_int + hum_dec + temp_int + temp_dec))
        {
            *humidity = hum_int;     // Stocker l'humidité entière
            *temperature = temp_int; // Stocker la température entière
        }
    }
}
```

Figure 3 : Code source des fonctions DHT11_ReadByte et DHT11_ReadData

Le tableau suivant résume les durées théoriques des différentes phases du protocole DHT11 :

Phase	Durée	Description
Signal de démarrage (état bas)	≥ 18 ms	Généré par le STM32
Relâchement de la ligne	20–40 µs	STM32 relâche la ligne
Réponse capteur (état bas)	80 µs	Accusé de réception du DHT1
Réponse capteur (état haut)	80 µs	Préparation à l'envoi des données
Début de bit (état bas)	50 µs	Identique pour chaque bit
Bit '0' (état haut)	26–28 µs	Durée courte = bit logique '0'
Bit '1' (état haut)	70 µs	Durée longue = bit logique '1'

La figure suivante présente le chronogramme théorique du protocole de communication du DHT11 tel que défini dans la datasheet. Elle sert de référence pour comparer les mesures expérimentales réalisées au PicoScope:

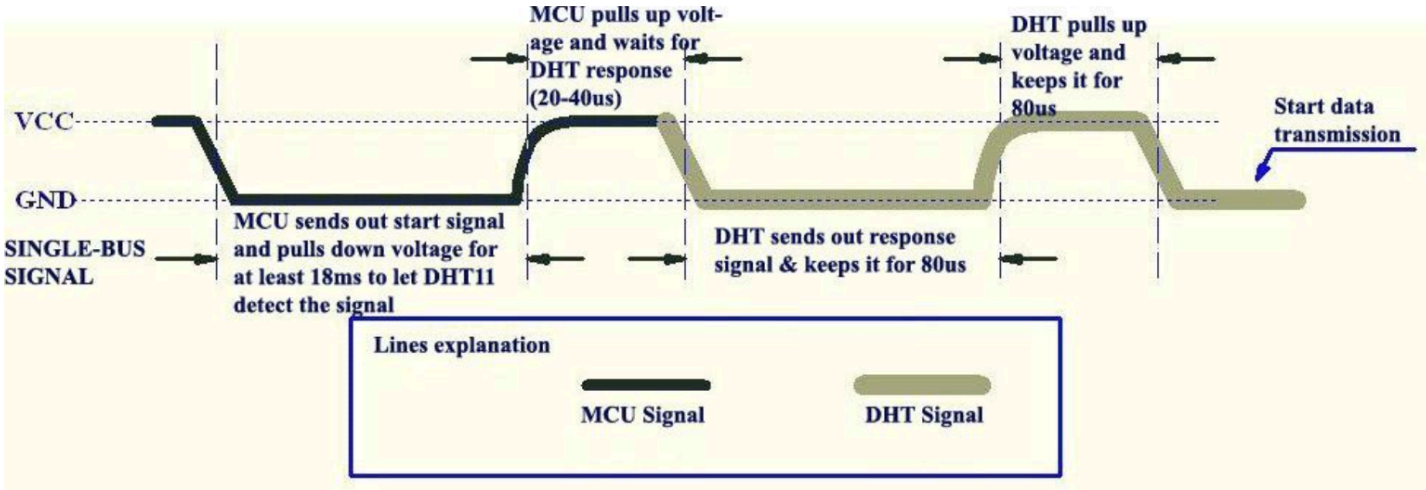

Figure 4: Chronogramme du capteur DHT11 selon le datasheet

II.4 Affichage sur l'écran LCD I2C

Une fois les données d'humidité et de température extraites et vérifiées avec le checksum, on les formate et on les envoie à l'écran LCD Grove RGB via le bus I2C. L'écran Grove LCD communique via deux adresses I2C distinctes : une pour le contrôleur de texte et une pour le contrôleur du rétroéclairage RGB. La bibliothèque lcd.c / lcd.h s'occupe d'initialiser l'écran, de placer le curseur et d'afficher les valeurs mesurées.

III. Résultats Expérimentaux et Analyse

Dans cette partie, les signaux échangés entre le microcontrôleur STM32 et le capteur DHT11 ont été analysés à l'aide d'un oscilloscope PicoScope afin de valider expérimentalement le protocole de communication et de comparer les résultats obtenus aux spécifications de la datasheet.

III.1 Validation de l'Impulsion de Démarrage

La première étape de validation consiste à vérifier que le signal de démarrage généré par le STM32 respecte la durée minimale de 18 ms.

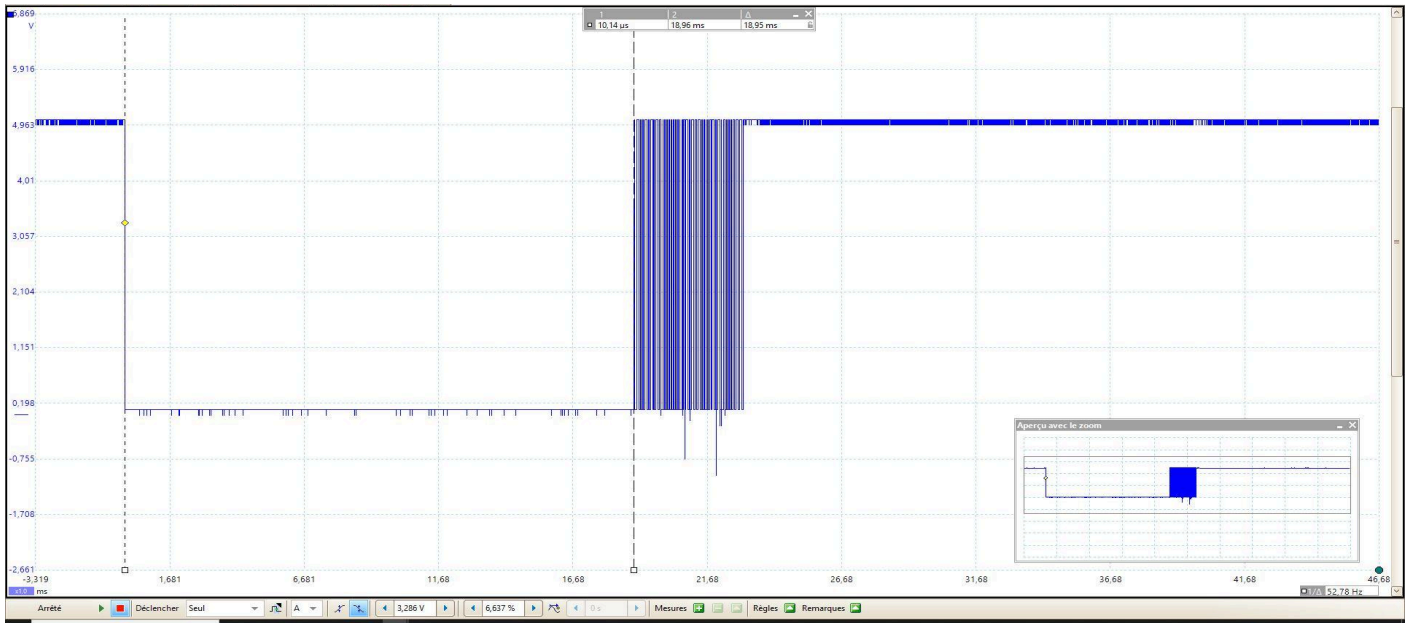

Figure 5 : Chronogramme de la communication DHT11 (PicoScope, 5 ms/div)

Les mesures effectuées à l'aide du PicoScope montrent une impulsion de durée d'environ 18,95 ms, ce qui est conforme à la spécification minimale requise. Ce résultat valide le bon fonctionnement de la phase d'initialisation de la communication.

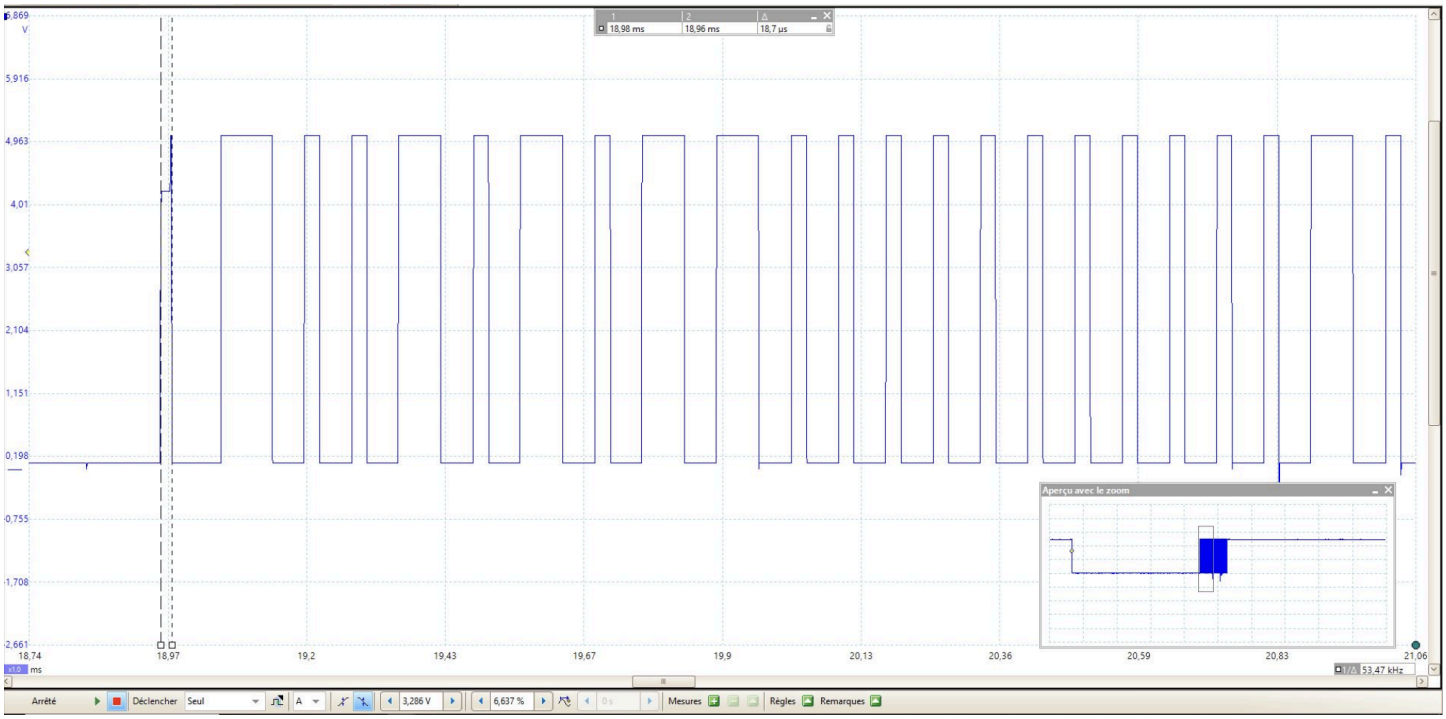

Figure 6 : Détail du relâchement de la ligne après le signal de démarrage

Cette figure met indique la phase de relâchement de la ligne de données par le STM32 après l'impulsion de démarrage. La durée mesurée est d'environ 18,7 µs, ce qui reste cohérent avec la plage théorique de 20 à 40 µs indiquée dans la datasheet.

III.2 Analyse de la Trame du Capteur (40 bits)

Après la phase d'initialisation, le DHT11 transmet la trame de 40 bits. La distinction entre les bits '0' et '1' est clairement visible sur l'oscillogramme par la différence de durée des impulsions à l'état haut.

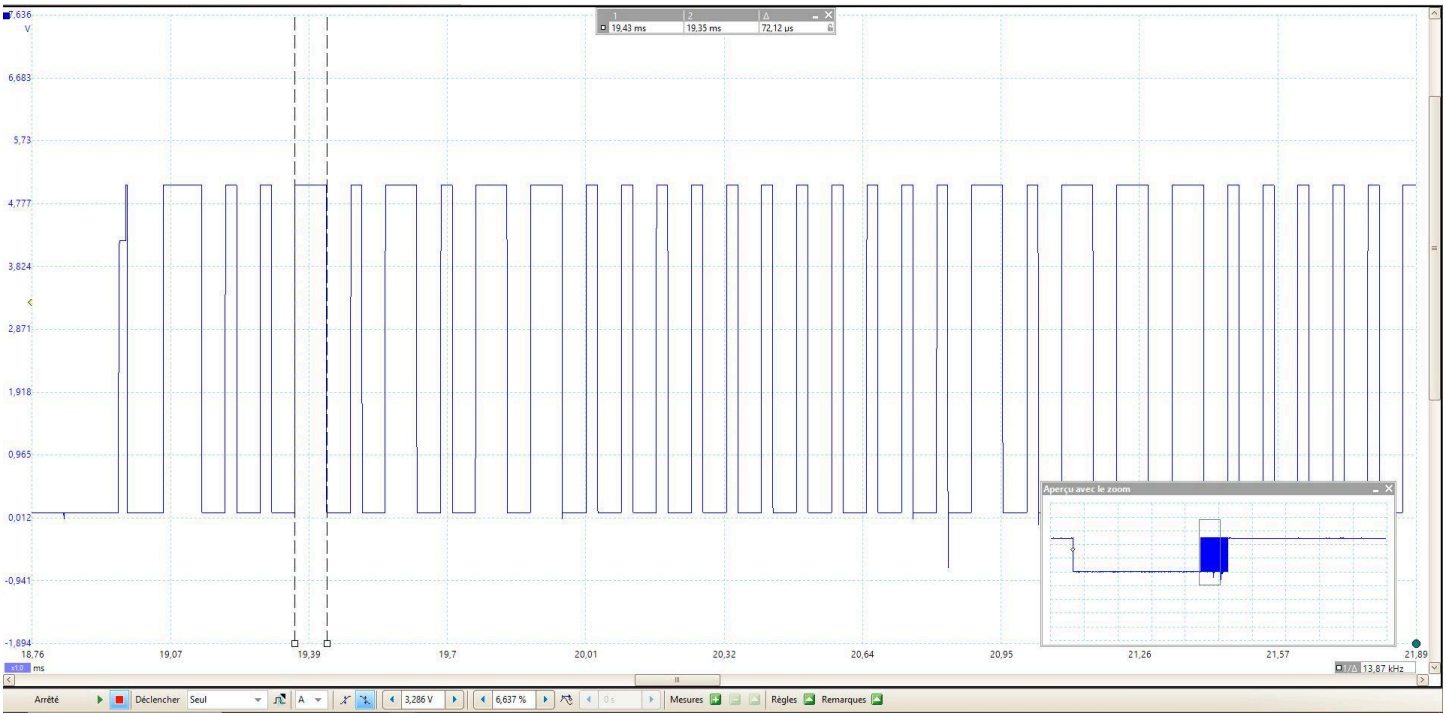

Figure 7 : Détail des impulsions de données “Etat haut”

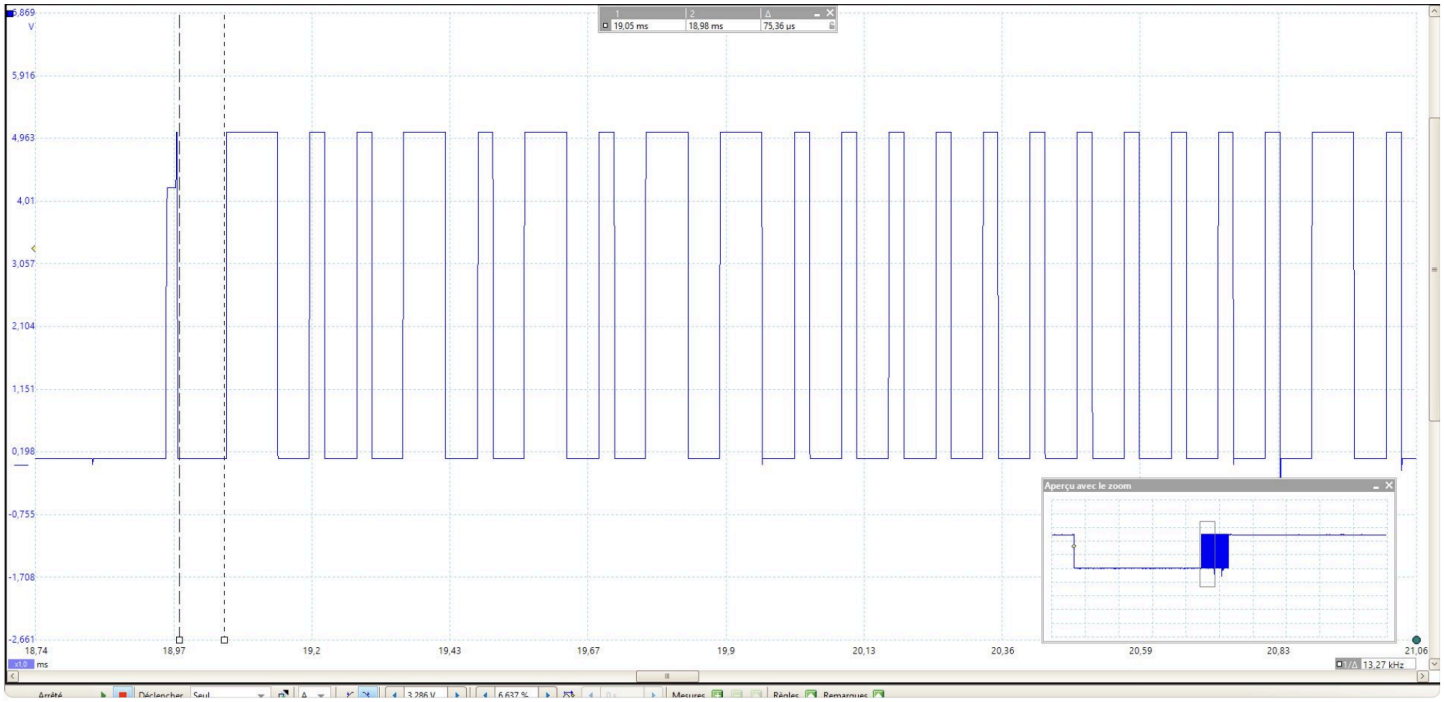

Figure 8 : Détail des impulsions de données “Etat bas”

Les mesures PicoScope montrent une durée d’impulsion haute d’environ **72 µs** et une durée d’impulsion basse d’environ **75 µs**, légèrement inférieures aux **80 µs** théoriques annoncés dans la documentation.

Conclusion

Ce TP nous a permis de mettre en œuvre un système d'acquisition de température et d'humidité basé sur le capteur DHT11 et la carte STM32. La partie la plus délicate était la gestion des temporisations à la microseconde, résolue grâce au timer TIM6. Les mesures réalisées au PicoScope ont montré que les signaux générés correspondent bien au protocole du DHT11, avec une impulsion de démarrage d'environ 18,95 ms, conforme à la spécification minimale de 18 ms. Ce projet nous a donné une meilleure compréhension de la communication numérique bas niveau et de la programmation embarquée sur STM32.